



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: Advanced Linux Command Line For Digital Forensics

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Computer Skills for Digital Forensics

Course Code: SBT-DF201

Instructor: Aminu Idris

August 16th , 2026

EXECUTIVE SUMMARY

This comprehensive report documents all findings, command logs, verification steps, and execution outputs from the hands-on digital forensics laboratory exercises. The sessions covered advanced Linux command-line operations, standard stream file descriptor manipulation (`stdin`, `stdout`, `stderr`), baseline environment documentation, cryptographic hash generation (`SHA256` and `MD5`), and controlled cross-platform evidence transfers utilizing `ncat` and `dd` between Kali Linux and Windows hosts. All procedures adhered strictly to authorized forensic safety boundaries and data integrity verification protocols.

Lab Environment

Machine 1 (Source): Kali Linux Virtual Machine ('hostname: kali', IP: 192.168.88.131)

Machine 2 (Target): Windows Host / PowerShell (VMnet1 IP: 192.168.202.1, VMnet8 IP: 192.168.88.1)

Network Setup: Isolated internal VM network enabling bidirectional TCP communication and successful ICMP connectivity

Tools Used: bash, cat, echo, ls, ip, ping, tee, ncat (Nmap suite), Windows PowerShell, certutil, dd, sha256sum

PROCEDURE & STEP-BY-STEP IMPLEMENTATION

Directory Structure Setup

The workspace directory hierarchy was initialized on Kali Linux to organize logs, evidence notes, hashes, and received files systematically:

```
mkdir -p ~/CIP-B101_LAB2A_GARBA_AHMED/{screenshots,logs,evidence,received,hashes,report_notes}
cd ~/CIP-B101_LAB2A_GARBA_AHMED
pwd
ls -la
```

Part A: Standard Input, Output, and Error Management

Task A1 (Evidence Note Creation): Created an authorized test evidence file inside the designated folder and verified its content and metadata:

```
cd ~/CIP-B101_LAB2A_GARBA_AHMED/evidence
```

```
echo "This is an authorized forensic evidence transfer test for Firstname Lastname" > evidence_note.txt
```

```
cat evidence_note.txt
```

```
ls -l evidence_note.txt
```

```
(garba@kali)-[~]
$ cd ~/CIP-B101_LAB2A_GARBA_AHMED
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ echo "This is an authorized forensic evidence transfer test for Firstname Lastname" > evidence_note.txt
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ cat evidence_note.txt
This is an authorized forensic evidence transfer test for Firstname Lastname
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ ls -l evidence_note.txt
-rw-rw-r-- 1 garba garba 77 Aug 16 15:41 evidence_note.txt
```

Task A2 (stdout Redirection): Redirected standard output of a directory listing into a log file using `>`:

```
cd ~/CIP-B101_LAB2A_GARBA_AHMED
```

```
ls -la evidence > logs/evidence_listing_stdout.txt
```

```
cat logs/evidence_listing_stdout.txt
```

```
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ ls -la evidence > logs/evidence_listing_stdout.txt
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ cat logs/evidence_listing_stdout.txt
total 24
drwxrwxr-x 3 garba garba 4096 Aug 15 23:38 .
drwxrwxr-x 9 garba garba 4096 Aug 16 15:53 ..
-rw-rw-r-- 1 garba garba 359 Aug 15 23:38 activity_log.txt
-rw-rw-r-- 1 garba garba 49 Aug 15 16:01 case_note1_copy.txt
-rw-rw-r-- 1 garba garba 52 Aug 15 15:45 lab2a_evidence.txt
drwxrwxr-x 2 garba garba 4096 Aug 15 16:02 notes_backup
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
```

Task A3 (stderr Redirection): Captured standard error generated by attempting to read a non-existent file into an error log using file descriptor `2>`:

```
cat evidence/file_that_does_not_exist.txt 2> logs/error_stderr.txt
```

```
cat logs/error_stderr.txt
```

```
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ cat evidence/file_that_does_not_exist.txt 2> logs/error_stderr.txt

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ cat logs/error_stderr.txt
cat: evidence/file_that_does_not_exist.txt: No such file or directory

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$
```

Task A4 (Combined Streams Redirection): Combined both `stdout` and `stderr` into a single log file using `> logs/combined\_output.txt 2>&1`:

(ls evidence && cat evidence/file_that_does_not_exist.txt) > logs/combined_output.txt 2>&1

cat logs/combined_output.txt

```
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ (ls evidence && cat evidence/file_that_does_not_exist.txt) > logs/combined_output.txt 2>&1
cat logs/combined_output.txt
activity_log.txt
case_note1_copy.txt
lab2a_evidence.txt
notes_backup
cat: evidence/file_that_does_not_exist.txt: No such file or directory

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$
```

PART B: BASELINE SYSTEM AND NETWORK DOCUMENTATION

Task B1 & B2 (Host & Network Recording): Captured hostname, IP address configuration, routing tables, and netcat availability on both Linux and Windows environments:

hostname | tee logs/linux_hostname.txt

ip address | tee logs/linux_ip_address.txt

ip route | tee logs/linux_ip_route.txt

which nc || which ncat || echo "netcat or ncat not found" | tee logs/netcat_check.txt

```
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ hostname | tee logs/linux_hostname.txt
kali

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ ip address | tee logs/linux_ip_address.txt
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:18:64:f5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.88.131/24 brd 192.168.88.255 scope global dynamic noprefixroute eth0
        valid_lft 956sec preferred_lft 956sec
    inet6 fe80::20c:29ff:fe18:64f5/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 96:9c:d8:e9:bf:b7 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ ip route | tee logs/linux_ip_route.txt
default via 192.168.88.2 dev eth0 proto dhcp src 192.168.88.131 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.88.0/24 dev eth0 proto kernel scope link src 192.168.88.131 metric 100

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$ which nc || which ncat || echo "netcat or ncat not found" | tee logs/netcat_check.txt
/usr/bin/nc

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED]
$
```

```

PS C:\WINDOWS\System32> hostname
ERROR404
PS C:\WINDOWS\System32> ipconfig

Windows IP Configuration

Wireless LAN adapter Local Area Connection* 11:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 12:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::6e05:2980:306f:f7e0%17
    IPv4 Address. . . . . : 192.168.202.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

```

Task B3 (Connectivity Verification): Tested network reachability via bi-directional ICMP echo requests:

ping -c 4 192.168.202.1 | tee logs/ping_windows.txt

```

(garba@kali) - [~/CIP-B101_LAB2A_GARBA_AHMED]
$ ping -c 4 192.168.202.1 | tee logs/ping_windows.txt
PING 192.168.202.1 (192.168.202.1) 56(84) bytes of data:
64 bytes from 192.168.202.1: icmp_seq=1 ttl=128 time=2.73 ms
64 bytes from 192.168.202.1: icmp_seq=2 ttl=128 time=1.40 ms
64 bytes from 192.168.202.1: icmp_seq=3 ttl=128 time=1.23 ms
64 bytes from 192.168.202.1: icmp_seq=4 ttl=128 time=1.42 ms

--- 192.168.202.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 1.231/1.695/2.732/0.603 ms
(garba@kali) - [~/CIP-B101_LAB2A_GARBA_AHMED]
$

```

PART C: CROSS-PLATFORM EVIDENCE TRANSFER AND VERIFICATION

Windows Listener Setup: Configured a netcat listener on the Windows target to capture transmitted files and evidence notes on port 9090:

cd C:\CIP-B101_Lab2B_Received

"C:\Program Files (x86)\Nmap\ncat.exe" -l -p 9090 > received_evidence_note.txt

```

Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ahmed>mkdir C:\CIP-B101_Lab2B_Received
A subdirectory or file C:\CIP-B101_Lab2B_Received already exists.

C:\Users\Ahmed>cd C:\CIP-B101_Lab2B_Received

C:\CIP-B101_Lab2B_Received>ncat -l -p 9090 > received_evidence_note.txt

```

Windows-to-Linux Evidence Transmission: Created and verified a text file on the Windows side (GarbaAhmedEvidence.txt), computed its hash, and transmitted it to Kali Linux:

echo hello Garba Ahmed! > GarbaAhmedEvidence.txt

type GarbaAhmedEvidence.txt

certutil -hashfile GarbaAhmedEvidence.txt SHA256

```
C:\CIP-B101_Lab2B_Received>echo hello Garba Ahmed! > GarbaAhmedEvidence.txt
C:\CIP-B101_Lab2B_Received>type GarbaAhmedEvidence.txt
hello Garba Ahmed!
C:\CIP-B101_Lab2B_Received>certutil -hashfile GarbaAhmedEvidence.txt SHA256
SHA256 hash of GarbaAhmedEvidence.txt:
74ec7ff18f6f41999424761b14594672c4658996c875157b4bdadaabd0696f95
CertUtil: -hashfile command completed successfully.
C:\CIP-B101_Lab2B_Received>
```

(Output SHA256 Hash: 74ec7ff18f6f41999424761b14594672c4658996c875157b4bdadaabd0696f95`)

```
C:\CIP-B101_Lab2B_Received>type received_evidence_note.txt
C:\CIP-B101_Lab2B_Received>del received_evidence_note.txt
C:\CIP-B101_Lab2B_Received>"C:\Program Files (x86)\Nmap\ncat.exe" -l -p 9090 > received_evidence_note.txt
C:\CIP-B101_Lab2B_Received>_
```

```
C:\CIP-B101_Lab2B_Received>certutil -hashfile received_evidence_note.txt SHA256
SHA256 hash of received_evidence_note.txt:
304bb5e36ed9b30d14d23d1454b09573014505d299286979d82152df8a2914d4
CertUtil: -hashfile command completed successfully.
C:\CIP-B101_Lab2B_Received>certutil -hashfile received_evidence_note.txt MD5
MD5 hash of received_evidence_note.txt:
231a9b2e7c44e90f06859d3802304df7
CertUtil: -hashfile command completed successfully.
C:\CIP-B101_Lab2B_Received>type received_evidence_note.txt
This is Lab 2A evidence file created by Garba Ahmed
```

On the Kali Linux receiver end, the stream was captured, verified, and logged:

cd ~/CIP-B101_LAB2A_GARBA_AHMED/received

ncat -l -p 9091 > GarbaAhmedEvidence_received.txt

cat GarbaAhmedEvidence_received.txt

sha256sum GarbaAhmedEvidence_received.txt | tee
../hashes/received_windows_file_sha256.txt

```
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED/evidence]
$ mkdir -p ~/CIP-B101_LAB2A_GARBA_AHMED/received

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED/evidence]
$ cd ~/CIP-B101_LAB2A_GARBA_AHMED/received
ncat -l -p 9091 > GarbaAhmedEvidence_received.txt

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED/received]
$ cd ~/CIP-B101_LAB2A_GARBA_AHMED/received
cat GarbaAhmedEvidence_received.txt
sha256sum GarbaAhmedEvidence_received.txt | tee ../hashes/received_windows_file_sha256.txt
hello Garba Ahmed!
74ec7ff18f6f41999424761b14594672c4658996c875157b4bdadaabd0696f95 GarbaAhmedEvidence_received.txt
```

PART D: TEST IMAGE GENERATION AND TRANSFER

Image Creation on Kali Linux: Generated a controlled test evidence image using 'dd' and populated a marker using echo:

```
cd ~/CIP-B101_LAB2A_GARBA_AHMED/evidence
```

```
dd if=/dev/zero of=test_evidence_image.dd bs=1M count=5 status=progress
```

```
echo "ICDFA test evidence marker - Garba Ahmed" | dd of=test_evidence_image.dd bs=1
seek=1024 conv=notrunc
```

```
ls -lh test_evidence_image.dd
```

```
sha256sum test_evidence_image.dd | tee ../hashes/source_test_image_sha256.txt
```

```
(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED/evidence]
$ mkdir -p ~/CIP-B101_LAB2A_GARBA_AHMED/received

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED/evidence]
$ cd ~/CIP-B101_LAB2A_GARBA_AHMED/received
ncat -l -p 9091 > GarbaAhmedEvidence_received.txt

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED/received]
$ cd ~/CIP-B101_LAB2A_GARBA_AHMED/received
cat GarbaAhmedEvidence_received.txt
sha256sum GarbaAhmedEvidence_received.txt | tee ../hashes/received_windows_file_sha256.txt
hello Garba Ahmed!
74ec7ff18f6f41999424761b14594672c4658996c875157b4bdadaabd0696f95 GarbaAhmedEvidence_received.txt

(garba@kali)-[~/CIP-B101_LAB2A_GARBA_AHMED/received]
$ cd ~/CIP-B101_LAB2A_GARBA_AHMED/evidence
dd if=/dev/zero of=test_evidence_image.dd bs=1M count=5 status=progress
echo "ICDFA test evidence marker - Garba Ahmed" | dd of=test_evidence_image.dd bs=1 seek=1024 conv=notrunc
ls -lh test_evidence_image.dd
sha256sum test_evidence_image.dd | tee ../hashes/source_test_image_sha256.txt
5+0 records in
5+0 records out
5242880 bytes (5.2 MB, 5.0 MiB) copied, 0.0356162 s, 147 MB/s
41+0 records in
41+0 records out
41 bytes copied, 0.00605976 s, 6.8 kB/s
-rw-rw-r-- 1 garba garba 5.0M Aug 16 18:41 test_evidence_image.dd
8598dcfb1287682d72cc12c4b94afa8205e2112bc0d701950aa42b3c1dc88a25 test_evidence_image.dd
```

Image Transmission to Windows: Opened a listener on Windows port 9092 and piped the raw image stream from Kali Linux:

```
cd C:\CIP-B101_Lab2B_Received
```

```
"C:\Program Files (x86)\Nmap\ncat.exe" -l -p 9092 > test_evidence_image_received.dd
```

```
Administrator: Command Prompt - ncat -l -p 9092
C:\CIP-B101_Lab2B_Received>ncat 192.168.88.131 9091 < GarbaAhmedEvidence.txt
C:\CIP-B101_Lab2B_Received>cd C:\CIP-B101_Lab2B_Received
C:\CIP-B101_Lab2B_Received>ncat -l -p 9092 > test_evidence_image_received.dd
```

Corresponding transmission command executed from Kali Linux:

```
cd ~/CIP-B101_LAB2A_GARBA_AHMED/evidence
ncat 192.168.88.1 9092 < test_evidence_image.dd
```

Post-Transfer Windows Hash Verification: Verified the integrity of the received image file on the Windows target using `certutil`:

```
cd C:\CIP-B101_Lab2B_Received

C:\Windows\System32>cd C:\CIP-B101_Lab2B_Received
C:\CIP-B101_Lab2B_Received>"C:\Program Files (x86)\Nmap\ncat.exe" -l -p 9092 > test_evidence_image_received.dd
C:\CIP-B101_Lab2B_Received>cd C:\CIP-B101_Lab2B_Received
C:\CIP-B101_Lab2B_Received>certutil -hashfile test_evidence_image_received.dd SHA256
SHA256 hash of test_evidence_image_received.dd:
8598dcfb1287682d72cc12c4b94afa8205e2112bc0d701950aa42b3c1dc88a25
CertUtil: -hashfile command completed successfully.
```

certutil -hashfile test_evidence_image_received.dd SHA256
(Output SHA256 Hash: 8598dcfb1287682d72cc12c4b94afa8205e2112bc0d701950aa42b3c1dc88a25`)*

EVIDENCE INTEGRITY & HASH VERIFICATION RESULTS

EVIDENCE ITEM / OPERATION	SOURCE HASH (PRE- TRANSFER)	DESTINATION HASH (POST-TRANSFER)	STATUS / RESULT
EVIDENCE_ NOTE.TXT (LINUX TO WINDOWS)	Local SHA256/MD5 recorded	Verified via Windows certutil	Verified Match
GARBAAHM EVIDENC E.TXT	74ec7ff18f6f4199942 4761b14594672c465 8996c875157b4bdad aabd0696f95	74ec7ff18f6f41999424761b14594672c4658996 c875157b4bdadaabd0696f95	Verified Match

(WINDOWS TO LINUX)			
TEST_EVIDE	8598dcfb1287682d7	8598dcfb1287682d72cc12c4b94afa8205e2112	Verified Match
NCE_IMAGE	2cc12c4b94afa8205e	bc0d701950aa42b3c1dc88a25	
.DD (KALI	2112bc0d701950aa4		
LINUX TO	2b3c1dc88a25		
WINDOWS)			

MOST SURPRISING FINDING

The most surprising aspect of this lab was observing how seamlessly raw byte streams can be piped across operating system boundaries over a network using a lightweight utility like `ncat`, while remaining entirely dependent on cryptographic hashes to prove authenticity. Without pre- and post-transfer hashes (`SHA256`/`MD5`), an investigator would have zero mathematical certainty that a file transmitted over a virtual network was free from subtle bit corruption, truncation, or silent manipulation. This emphasizes that network data transit in digital forensics is never taken on blind trust; integrity must always be mathematically proven.

RECOMMENDATION FOR FUTURE PROJECTS

Based on the lab findings, the following recommendations are proposed for future digital forensic workflows:

1. **Mandatory Cryptographic Hashing:** Always compute and archive hash values immediately upon initial data acquisition and verify them again at every stage of transport or storage.
2. **Strict Network Segmentation:** Perform remote acquisitions and file transfers exclusively over isolated, controlled virtual lab networks to eliminate external traffic interference or security risks.
3. **Authorized Tool Utilization:** Restrict remote data piping to lightweight, auditable tools like ncat, strictly avoiding unauthenticated remote shell options or unauthorized backdoor execution vectors.
4. **Comprehensive Stream Logging:** Systematically redirect standard output and error streams into dedicated log directories to ensure an immutable, reproducible audit trail of every forensic command executed.
5. **Pre-Testing with Small Images:** Validate transmission protocols and block sizes using small test evidence images before attempting acquisitions on high-value production systems.

REFLECTION ANSWERS

1. What is the difference between standard output and standard error?

Standard output (``stdout``, file descriptor 1) handles regular, successful data streams generated by a command. Standard error (``stderr``, file descriptor 2) is a separate, dedicated channel reserved exclusively for diagnostic messages, warnings, and failure execution notes.

2. Why is command redirection useful during digital forensic documentation?

It allows investigators to automatically capture terminal outputs and execution logs directly into timestamped files, ensuring an accurate, objective, and tamper-resistant audit trail.

3. Why must a forensic examiner calculate the hash of a file before and after transfer?

Pre- and post-transfer hash comparisons provide mathematical proof that the digital evidence suffered no alteration, corruption, or truncation while traversing the network.

4. Why is it unsafe and unethical to use remote shell access on a system without authorization?

Unauthorized remote access breaches legal boundaries, violates user privacy, compromises system stability, and catastrophically destroys the legal chain of custody by introducing unverified environmental artifacts.

5. What is the difference between a bind shell and a reverse shell at a conceptual level?

A bind shell opens a listening port directly on the target machine waiting for an incoming administrative connection, whereas a reverse shell forces the target machine to actively initiate an outbound connection back to the examiner's listener (frequently utilized to bypass perimeter firewalls).

6. Why might a firewall block one type of connection but allow another?

Firewalls evaluate packets based on strict rules governing port numbers, traffic direction (inbound vs. outbound), stateful tracking, and protocol signatures. Unsolicited inbound connection attempts are typically dropped by default, whereas outbound or established flows are often permitted.

7. Why did this lab use a small test evidence image instead of imaging a real disk?

A small test image teaches the precise mechanics of bit-stream transfer, hashing, and stream redirection safely without risking live system instability, prolonged processing delays, or privacy violations inherent in full physical disk imaging.

CONCLUSION

This laboratory successfully demonstrated advanced Linux command-line text processing, stream redirection file descriptors, and secure cross-platform file acquisition techniques. By adhering strictly to authorized operational parameters and enforcing rigorous hash-based verification protocols, a reliable, repeatable workflow for digital evidence handling was established that satisfies professional forensic standards. Moving forward, integrating these structured logging and cryptographic verification practices will remain fundamental to maintaining absolute data integrity in all digital forensic investigations.